

École nationale de la statistique et de l'analyse de l'information

ENSAI — Bruz

Projet traitement de données

1^{re} Année — 2025–2026

Rapport du projet de traitement de données

*Application en ligne de commande
pour l'exploration de données sportives — Python*

Auteurs :

Mohammed-Adam LAKHRISSE

Jade COLIN

Mathias DOUILLARD

Mohamed Fouad CHAABANE

Coordinateur : Clément VALOT

Projet de traitement de données — 1^{re} année

Mai 2026

0 Résumé

Ce rapport présente le fonctionnement d'une application en ligne de commande (CLI) développée en Python, permettant d'explorer et d'analyser des données statistiques issues de neuf disciplines sportives et e-sportives : Basketball (NBA), Football (UEFA Champions League et ligues européennes), Tennis (ATP/WTa), League of Legends (EMEA), Counter-Strike 2, Échecs (FIDE), Volleyball (JO Paris 2024), StarCraft II et Badminton (BWF).

L'application repose sur une architecture modulaire organisée en quatre couches : les *Parsers* chargent et normalisent les fichiers CSV, le *Model* fournit des classes génériques (**Joueur**, **Equipe**, **Match**, **Sport**) paramétrables pour chaque sport, les modules d'*Analysis* implémentent les fonctionnalités statistiques spécifiques, et le *CLI* orchestre la navigation, les filtres et l'export CSV.

Le projet totalise environ **11 400 lignes de code Python** réparties sur **69 fichiers**, exploite **36 fichiers CSV** de données couvrant plus de **3 000 joueurs** et **30 000 matchs**, et dispose d'une suite de près de **3 800 lignes de tests unitaires** utilisant pytest. Après avoir présenté le cahier des charges et les choix d'architecture, nous détaillerons la gestion des données, la structure du code, l'utilisation de la programmation orientée objet et la répartition des rôles, avant de présenter les résultats obtenus.

Mots-clés : Python, pandas, traitement de données, sports, CLI, programmation orientée objet, tests unitaires.

0 Table des matières

1	Introduction	1
2	Cahier des charges	1
2.1	Diagramme de cas d'utilisation	2
2.2	Diagramme de classes	3
2.3	Sports couverts et classification	3
2.4	Fonctionnalités par sport	4
2.5	Architecture générale	4
3	Répartition des rôles	4
4	Gestion des données	6
4.1	Sources et formats des données	6
4.2	Chargement et normalisation (Parsers)	6
4.3	Nettoyage et validation	7
5	Structure du code	7
5.1	Organisation des packages	7
5.2	Le fichier <code>__main__.py</code>	8
5.3	Le moteur de navigation CLI	8
5.4	Le module générique (<code>générique.py</code>)	9
6	Programmation orientée objet	10
6.1	La classe Sport et les enums de classification	10
6.2	Les classes Joueur et Joueurs	10
6.3	Les classes Equipe et Equipes	11
6.4	Les classes Match et Matches	11
6.5	La classe abstraite StatsSport	11
6.6	Le pattern BaseLoader	11
7	Tests et validation	12
7.1	Stratégie de test	12
7.2	Couverture	13
7.3	Formatage et linting	13
8	Résultats	13
8.1	Scénario 1 : Navigation et filtrage des sports	13
8.2	Scénario 2 : Fiche joueur avec sélecteur paginé	14
8.3	Scénario 3 : Bracket de tournoi	15

8.4	Scénario 4 : Statistiques d’équipe e-sport	15
8.5	Scénario 5 : Export CSV	16
9	Conclusion	16
	Annexes	19
	Annexe A — Bilan personnel de Jade Colin	19
	Annexe B — Bilan personnel de Mathias Douillard	19
	Annexe C — Bilan personnel de Mohammed-Adam Lakhri	20
	Annexe D — Bilan personnel de Mohamed Fouad Chaabane	20

1 Introduction

Suite à l’engouement suscité par les Jeux Olympiques et Paralympiques de Paris 2024, le Ministère des Sports, de la Jeunesse et de la Vie Associative a exprimé le besoin de se doter d’un outil informatique performant pour centraliser et gérer les résultats de diverses compétitions sportives.

L’objectif principal de notre travail a été de développer une application générique, permettant de modéliser, d’organiser et d’exploiter des entités universelles au monde du sport : des joueurs, des équipes, des matchs et des compétitions. Cette application permet à des utilisateurs d’accéder aux statistiques de ces compétitions, de les manipuler et d’en déduire des analyses complexes.

La complexité de cette mission réside dans la diversité des disciplines traitées. Notre application ne se limite pas à un seul sport, mais a été conçue pour intégrer aussi bien des sports collectifs classiques comme le football ou le basketball, que des disciplines individuelles comme le tennis ou les échecs, et même l’e-sport (League of Legends, Counter-Strike 2, StarCraft II). Chaque sport apportant ses propres spécificités statistiques, l’enjeu de ce projet est de taille : concevoir une application capable d’absorber des données hétérogènes tout en restant générique et évolutive, afin de pouvoir intégrer facilement de nouveaux sports ou de nouvelles sources sans remettre en cause son fonctionnement global.

Pour répondre à ces objectifs, nous avons adopté une démarche de **programmation orientée objet**, complétée par l’utilisation de la bibliothèque **pandas** pour optimiser le traitement de volumes de données importants. Nous avons construit nos données comme des objets pour garantir une modélisation fidèle du domaine sportif, tout en utilisant pandas pour assurer des performances élevées lors des calculs statistiques et la gestion des valeurs manquantes. Nous justifierons ces choix d’architecture et de programmation en détail dans la suite de ce document.

Les données proviennent de sources publiques variées (NBA API, ATP/WTN, Liquipedia, FIDE, données des JO Paris 2024, etc.), toutes normalisées au format CSV. Le projet a été développé collaborativement via Git et GitHub, en suivant une organisation modulaire qui facilite l’ajout de nouveaux sports.

2 Cahier des charges

L’application devait répondre à plusieurs exigences : couvrir au moins cinq sports différents, proposer des fonctionnalités d’analyse statistique pertinentes pour chaque sport, offrir une interface de navigation intuitive en ligne de commande, et être accompagnée de tests unitaires (pytest, black, Flake8, docstrings NumPy). Nous avons choisi de couvrir neuf

disciplines pour enrichir la comparaison entre sports traditionnels et e-sports.

2.1 Diagramme de cas d'utilisation

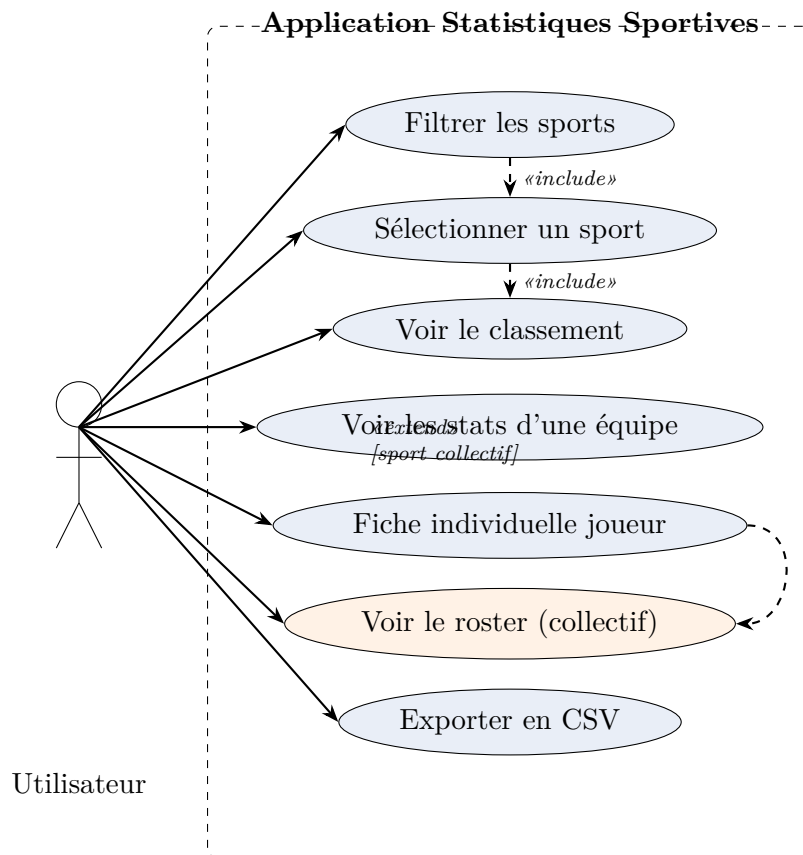


FIGURE 1 – Diagramme de cas d'utilisation.

Le diagramme de cas d'utilisation décrit les interactions entre l'utilisateur et l'application. L'utilisateur dispose de deux actions de navigation. Il peut d'abord **filtrer les sports** affichés en appliquant des critères comme le type de sport (individuel ou collectif), la catégorie (sport traditionnel ou e-sport) ou le format de compétition. Ce filtre inclut automatiquement l'étape suivante : la **sélection d'un sport** dans la liste résultante.

Une fois un sport sélectionné, l'utilisateur accède aux fonctionnalités d'analyse : le **classement**, les **statistiques d'équipe** ou la **fiche individuelle** d'un joueur. La fonctionnalité *voir le roster* est disponible uniquement pour les sports collectifs (basketball, LoL, CS2, football, volleyball) — c'est ce que représente la relation «**extend**» conditionnée à [sport collectif]. Les relations «**include**» indiquent des enchaînements obligatoires : filtrer les sports mène systématiquement à en sélectionner un, et sélectionner un sport donne nécessairement accès aux fonctionnalités d'analyse.

2.2 Diagramme de classes

Le diagramme ci-dessous présente l’architecture orientée objet du projet. Notre projet est structuré en trois couches principales.

La première est la couche **Model**, qui contient les classes représentant les données métier. On y trouve trois entités : **Joueur**, **Match** et **Equipe**. Chacune existe en double : une classe pour un objet individuel et une classe collection (par exemple **Joueurs**) qui regroupe tous les objets du même type et propose des méthodes de filtrage et de statistiques.

Ces entités sont liées entre elles : une **Equipe** peut récupérer la liste de ses joueurs à partir d’une collection **Joueurs**, et une collection **Matches** permet de calculer le bilan victoires/défaites d’une équipe donnée. La classe **Sport** est indépendante des autres : elle décrit simplement les caractéristiques d’un sport via trois énumérations, sans lien direct avec les joueurs ou les matchs.

La deuxième couche est **Parsers**. Elle contient la classe abstraite **BaseLoader** dont héritent les dix loaders, un par sport. Leur rôle est unique : charger les fichiers CSV et retourner des données exploitables par le reste de l’application.

La troisième couche est **Analysis**. Elle regroupe les modules de calcul statistique propres à chaque sport. Ces modules s’appuient sur les loaders pour charger les données et sur les classes du Model pour les manipuler. C’est la classe **CLI** qui orchestre l’ensemble en appelant ces modules selon les choix de l’utilisateur.

2.3 Sports couverts et classification

Chaque sport est caractérisé par trois axes de classification, modélisés par des enums Python (**TypeSport**, **CategorieSport**, **TypeCompétition**) :

Sport	Type	Catégorie	Compétition
Basketball (NBA)	Collectif	Sport	Points
League of Legends	Collectif	E-sport	Points
Football Champions L.	Collectif	Sport	Mixte
Football (ligues EU)	Collectif	Sport	Points
Volleyball	Collectif	Sport	Mixte
CS2	Collectif	E-sport	Mixte
Tennis (ATP/WTB)	Individuel	Sport	Éliminatoire
Badminton (BWF)	Individuel	Sport	Éliminatoire
Échecs (FIDE)	Individuel	E-sport	Éliminatoire
StarCraft II	Individuel	E-sport	Éliminatoire

Cette classification permet à l’utilisateur de filtrer les sports dès le menu principal via une

logique ET combinant les trois critères. Les sports de type *Mixte* apparaissent dans les résultats des filtres *Points* et *Éliminatoire*.

2.4 Fonctionnalités par sport

Les fonctionnalités sont adaptées à la nature de chaque sport. Pour les sports **collectifs** (basketball, LoL, football, CS2, volleyball), l'utilisateur peut choisir entre des statistiques sur les **équipes** (classement, stats détaillées, bracket, roster) et sur les **joueurs** (fiche individuelle, liste paginée). Pour les sports **individuels** (tennis, échecs, badminton, StarCraft II), les fonctionnalités portent directement sur les joueurs.

Le volleyball et le tennis proposent en outre une distinction **hommes/femmes**, ajoutant un niveau de navigation supplémentaire. Parmi les fonctionnalités communes à tous les sports : la **fiche joueur** (informations personnelles, statistiques clés), les **classements** (par victoires, par Elo pour les échecs, par points), les **bilans** (victoires/défaites/nuls) et l'**export CSV** de tout résultat tabulaire.

2.5 Architecture générale

L'application s'organise en quatre couches principales. La figure 2 présente cette architecture, dont le diagramme de classes ci-après détaille les relations orientées objet.

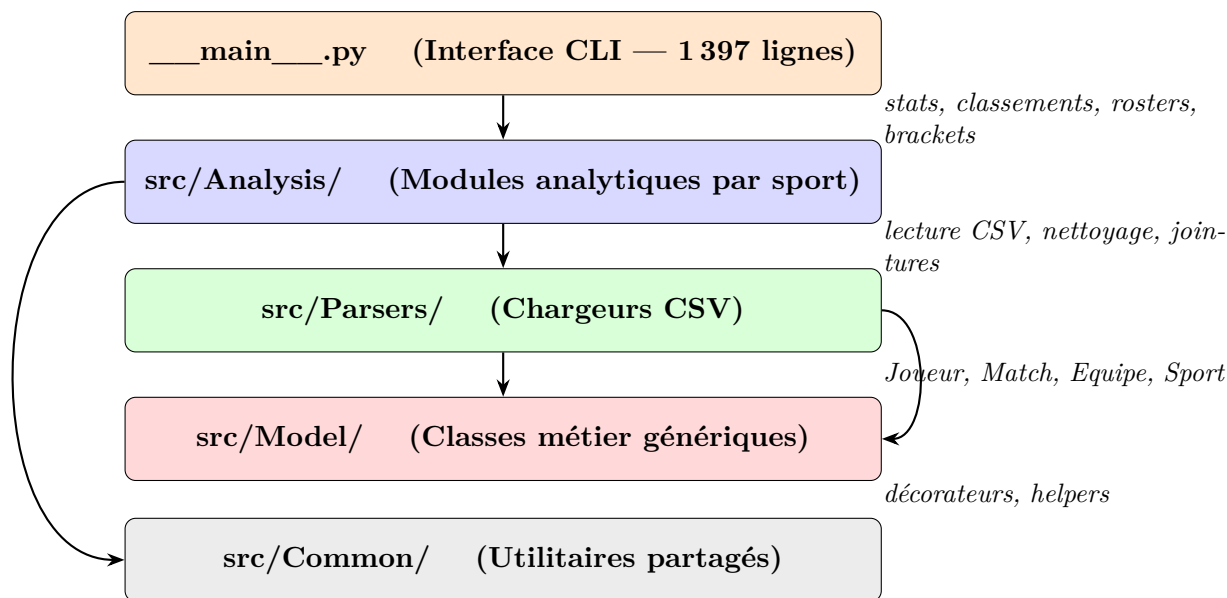


FIGURE 2 – Architecture en couches de l'application.

3 Répartition des rôles

Ce projet a suivi plusieurs phases principales. La conception a de loin été la phase la plus longue, en raison d'un emploi du temps chargé en cette période. Nous avons réfléchi

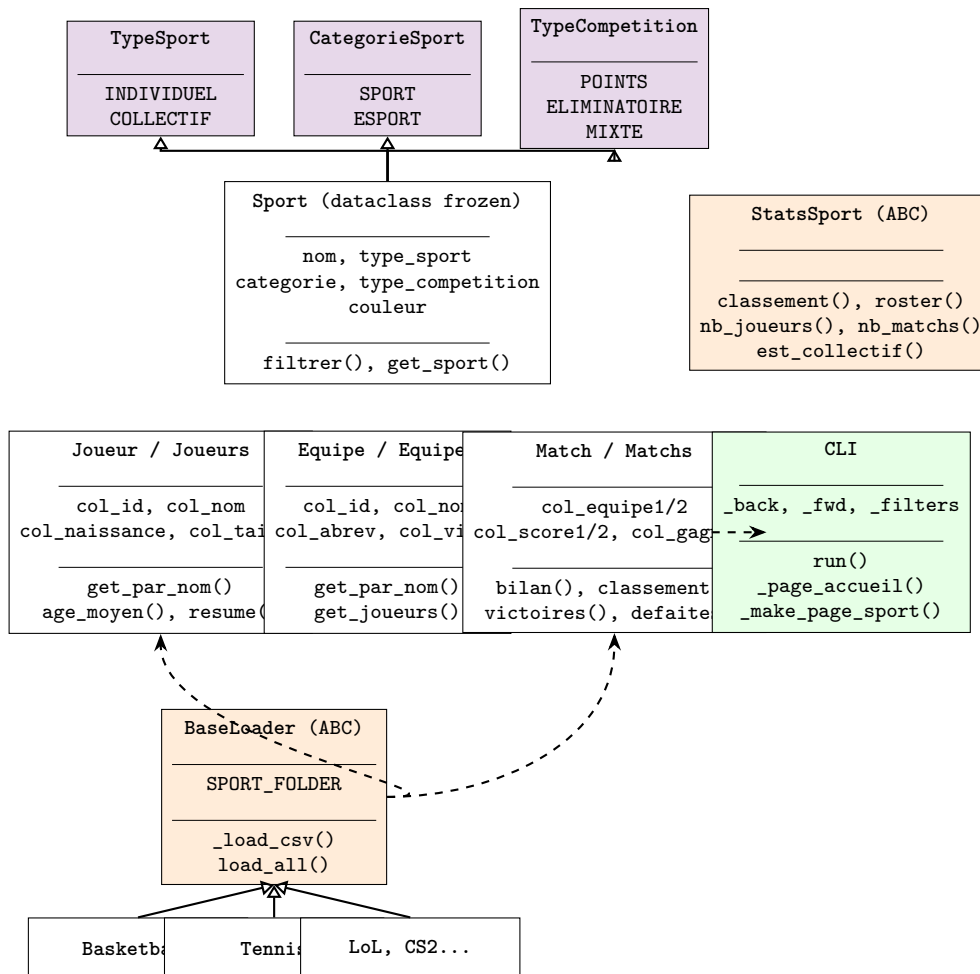


FIGURE 3 – Diagramme de classes simplifié de l'application.

ensemble à la structure globale de notre future application et il a d'abord fallu comprendre clairement les attendus du projet.

Phase de conception — Réflexion collective sur l'architecture, le choix des sports, la structure des données et l'organisation des modules. Chaque membre a prototypé ses idées dans un dossier individuel (Projet_Adam, Projet_Jade, Projet_Mathias, Projet_fouad) avant la fusion dans le code commun.

Phase de développement — Les tâches ont été réparties par couche et par sport. La couche Model (classes génériques Joueur, Equipe, Match) et le BaseLoader ont été développés en priorité pour fournir les fondations communes. Puis chaque membre a implémenté les modules d'analyse et les loaders de ses sports attribués. Le module générique a été développé itérativement au fur et à mesure que les patterns communs émergeaient.

Phase d'intégration — Le `__main__.py` et le moteur CLI ont été développés pour relier tous les modules. Cette phase a nécessité un travail de coordination important pour harmoniser les interfaces entre les modules d'analyse.

Phase de tests et documentation — Les tests unitaires ont été écrits en parallèle du développement, chaque membre testant ses modules. Le formatage du code est assuré par black. La documentation est intégrée via les docstrings Python (format NumPy).

Membre	Contributions principales
M.-A. LAKHRISSI	Architecture, Model, CLI, intégration
Jade COLIN	Modélisation, classes de données, tests
Mathias DOUILLARD	Loaders, analyse, données, export
M. F. CHAABANE	Interface, visualisation, analyse

4 Gestion des données

4.1 Sources et formats des données

Les données du projet proviennent de sources publiques variées et sont toutes stockées au format CSV dans le dossier `data/`, organisé par sport. Au total, le projet exploite **36 fichiers CSV** répartis sur 10 dossiers.

Sport	Fichiers CSV	Joueurs	Matches	Source
Basketball (NBA)	3	431	~1 200	NBA API
Football (ligues EU)	5	~11 000	25 979	Open Football
Football Champions L.	3	751	125	Kaggle
Tennis (ATP + WTA)	4	778	5 765	Tennis Abstract
League of Legends	4	50	45	Liquipedia
Échecs (FIDE)	2	206	256	FIDE
Volleyball (JO 2024)	7	311	~90	Paris 2024
CS2	4	160	106	Liquipedia
StarCraft II	2	32	67	Liquipedia
Badminton (BWF)	2	88	226	BWF
Total	36	>3 000	>30 000	

4.2 Chargement et normalisation (Parsers)

Le chargement des données est centralisé dans le package `src/Parsers`. Une classe abstraite **BaseLoader** fournit l’infrastructure commune : résolution du chemin vers le dossier de données, lecture CSV via pandas, et conversion automatique des colonnes de dates. Chaque sport hérite de cette classe en spécifiant simplement son `SPORT_FOLDER` et en implémentant ses méthodes `load_*`().

Par exemple, le **BasketballLoader** charge trois fichiers (`player.csv`, `team.csv`, `game.csv`)

et retourne un tuple de trois DataFrames. Le **VolleyballLoader**, plus complexe, charge sept fichiers en séparant joueurs et coachs par genre. Ce pattern permet d'ajouter un nouveau sport en quelques lignes.

```

1 class BadmintonLoader(BaseLoader):
2     SPORT_FOLDER = "badminton"
3
4     def load_all(self):
5         players = self._load_csv("player.csv")
6         matches = self._load_csv("match.csv", date_cols=["date"])
7         return players, matches

```

Listing 1 – Structure minimale d'un loader (exemple Badminton)

4.3 Nettoyage et validation

Le nettoyage des données s'effectue à deux niveaux.

Dans les Loaders : les colonnes de dates sont parsées en `datetime` pandas via `pd.to_datetime(errors=)` ce qui gère automatiquement les formats variés et les valeurs manquantes. Pour le Basketball NBA, les tailles au format américain ("6-8" pour 6 pieds 8 pouces) sont converties en centimètres :

```

1 def _height_to_cm(h: str) -> float | None:
2     try:
3         feet, inches = map(int, h.split("-"))
4         return round((feet * 12 + inches) * 2.54, 1)
5     except (ValueError, AttributeError):
6         return None

```

Listing 2 – Conversion de taille NBA (BasketballLoader.py)

Dans les modules d'analyse : les fonctions appliquent un `dropna()` ciblé sur les colonnes pertinentes avant tout calcul, et utilisent des valeurs de remplacement "N/A" pour l'affichage des fiches joueurs. Pour les colonnes numériques (kills, gold, assists, etc.), les valeurs manquantes sont gérées implicitement par les méthodes agrégées de pandas.

5 Structure du code

5.1 Organisation des packages

Le projet suit une organisation en packages Python claire et modulaire, totalisant environ **11 400 lignes de code Python** réparties sur **69 fichiers**.

```

Projet-1A-2026/
|-- __main__.py          # Point d'entree CLI (1 397 lignes)

```

```

|-- requirements.txt      # pandas, pytest, pytest-cov, black
|-- README.md
|-- data/                # 36 fichiers CSV (10 sports)
|-- src/
|   |-- Model/           # Classes generiques
|   |-- Parsers/         # Loaders par sport
|   |-- Analysis/        # Modules analytiques
|   '-- Common/          # Utilitaires partages
|-- test/                # ~3 800 lignes de tests
|-- Projet_Adam/         # Contributions individuelles
|-- Projet_Jade/
|-- Projet_Mathias/
'-- Projet_fouad/

```

Les sous-packages importent rarement les uns des autres directement. Les modules d’analyse importent uniquement leurs Loaders et le module générique. Le `__main__.py` est le seul fichier qui relie tous les modules ensemble, conformément aux bonnes pratiques de couplage faible.

5.2 Le fichier `__main__.py`

Ce fichier de **1 397 lignes** est le chef d’orchestre de l’application. Il contient deux éléments principaux :

La configuration des sports (`_make_sports_config()`) — un dictionnaire associant chaque sport à ses fonctionnalités disponibles, organisées par catégorie (équipe/joueurs) et par genre le cas échéant. Chaque fonctionnalité est décrite par un label, une fonction d’analyse à appeler, et optionnellement un sélecteur avec pagination et filtres.

La classe CLI — implémente un moteur de navigation basé sur des pages-fonctions (*callable*s). Chaque page retourne soit une nouvelle page (navigation avant), soit `'back'/'forward'/'quit'` pour la navigation historique, soit `None` pour rester sur place. Ce pattern simple mais puissant permet de construire des arbres de navigation arbitraires sans machine à états complexe.

5.3 Le moteur de navigation CLI

La classe **CLI** gère un **historique de navigation avec deux piles** (`_back` et `_fwd`), permettant à l’utilisateur de revenir en arrière ou d’avancer dans son parcours. Le flux passe par quatre niveaux :

1. **Page d’accueil** — affiche les sports filtrés et l’option de filtrage.

2. **Page sport** — routeur qui oriente vers la sélection genre, équipe/joueurs, ou directement les stats selon la configuration.
3. **Page stats** — liste les fonctionnalités disponibles.
4. **Page résultat** — exécute la fonction d'analyse et propose l'export CSV.

Pour les sélecteurs avec de nombreuses options (joueurs, équipes), un système de **pagination par 20 éléments** avec filtres (recherche par nom, catégorie, année de naissance) permet une navigation fluide même sur de grandes bases (ex. : 443 joueurs ATP, 11 000 joueurs dans les ligues européennes de football).

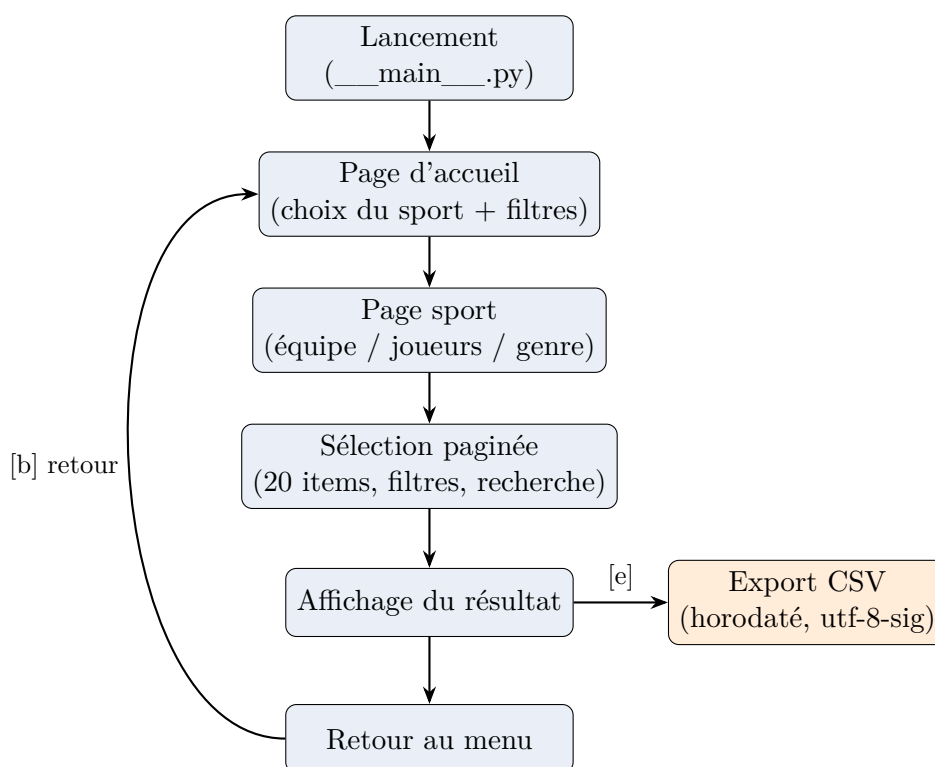


FIGURE 4 – Flux de navigation de l'interface CLI.

5.4 Le module générique (générique.py)

Ce module de **733 lignes** mutualise les fonctionnalités communes à tous les sports. Il fournit cinq fonctions principales :

- `formater_roster` — formate le roster d'une équipe avec les colonnes Rôle, Nom complet, Pseudo (e-sport uniquement), Nationalité, Date de naissance. Gère les coachs en tête de liste et les données manquantes.
- `afficher_bracket` — construit et affiche un tableau de tournoi éliminatoire en ASCII, avec les gagnants mis en évidence en vert (ANSI). Supporte les compétitions à aller-retour.

- `afficher_classement` — produit un classement par points (3 pts victoire, 1 pt nul, 0 pt défaite) avec séparateur de qualification optionnel.
- `fiche_joueur` — génère une fiche complète d'un joueur adaptée à chaque sport via un dictionnaire de labels.
- `lister_joueurs` — liste les joueurs d'une équipe ou d'un pays avec filtrage et tri alphabétique.

6 Programmation orientée objet

Le projet fait un usage mesuré mais pertinent de la programmation orientée objet. Comme le souligne le README, les classes sont principalement utiles pour « passer des données formatées à travers l'application » ; l'essentiel du traitement de données s'appuie sur les DataFrames pandas. Les classes génériques avec leur **mapping de colonnes paramétrable** offrent un bon compromis entre abstraction et simplicité.

6.1 La classe `Sport` et les enums de classification

La classe `Sport`, définie comme un `@dataclass(frozen=True)`, encapsule les métadonnées d'un sport : nom, type (collectif/individuel), catégorie (sport/e-sport), type de compétition (points/éliminatoire/mixte), et couleur d'interface. Trois enums structurent ces axes :

```

1 class TypeSport(Enum):
2     INDIVIDUEL = "individuel"
3     COLLECTIF = "collectif"
4
5 class CategorieSport(Enum):
6     SPORT = "sport"
7     ESPORT = "e-sport"
8
9 class TypeCompétition(Enum):
10     POINTS = "points"
11     ELIMINATOIRE = "éliminatoire"
12     MIXTE = "mixte"

```

Listing 3 – Enums de classification des sports (src/Model/sport.py)

Le registre `SPORTS` (liste de 9 instances) et la fonction `filtrer()` permettent de combiner les critères en logique ET pour le menu de filtrage du CLI. Des propriétés dérivées comme `est_en_points` gèrent la logique des sports de type Mixte.

6.2 Les classes `Joueur` et `Joueurs`

La classe `Joueur` enveloppe une `pd.Series` (ligne de DataFrame) avec un mapping de colonnes paramétrable : `col_id`, `col_nom`, `col_naissance`, `col_equipe`, `col_taille`. Des

propriétés calculées comme `age` dérivent automatiquement de la date de naissance via `pd.Timestamp.today()`.

La classe **Joueurs** (collection) wrappe un `DataFrame` entier et fournit des méthodes de filtrage (`get_par_nom`, `get_par_equipe`, `filtrer`) et de statistiques (`age_moyen`, `taille_moyenne`, `resume`). Elle implémente les protocoles Python standard (`__len__`, `__iter__`, `__getitem__`) pour une utilisation naturelle.

La compatibilité multi-sport est garantie par le paramétrage des colonnes à l’initialisation :

Sport	col_id	col_nom	col_equipe	col_taille
Basketball	person_id	full_name	team_id	height_cm
Tennis	player_id	full_name	—	height
LoL	pseudo	name	team	—
Volleyball	name	name	country_code	height

6.3 Les classes *Equipe* et *Equipes*

Suivant le même pattern que **Joueur/Joueurs**, ces classes modélisent les équipes avec les colonnes `col_id`, `col_nom`, `col_abrev`, `col_ville`. La méthode `get_joueurs(joueurs: Joueurs)` permet de récupérer les joueurs d’une équipe en croisant les deux collections — illustrant la composition orientée objet.

6.4 Les classes *Match* et *Matches*

La classe **Match** représente un match avec les colonnes d’équipes, de scores et de gagnant. La propriété `gagnant` implémente une logique à deux niveaux : elle utilise d’abord la colonne dédiée si elle existe, sinon déduit le gagnant depuis les scores. La collection **Matches** fournit des statistiques agrégées : `bilan`, `classement`, `score_moyen`, avec filtrage par équipe, date et saison.

6.5 La classe abstraite *StatsSport*

Cette ABC définit le contrat commun pour tous les modules analytiques : `classement(n)`, `nb_joueurs()`, `nb_matches()`, `stats_entite(nom)`. La méthode optionnelle `roster()` est disponible pour les sports collectifs. La méthode `est_collectif()` détecte automatiquement si `roster` a été surchargée via l’inspection Python.

6.6 Le pattern *BaseLoader*

BaseLoader est une ABC qui centralise la mécanique de chargement CSV. Elle implémente le **patron de conception Template Method** : les sous-classes n’ont qu’à déclarer

SPORT_FOLDER et à implémenter les méthodes `load_*`(), tout le reste étant géré par la classe mère.

```
1 class BaseLoader(ABC):
2     _DATA_ROOT: Path = Path(__file__).parent.parent.parent / "data"
3     SPORT_FOLDER: str = ""
4
5     @property
6     def root(self) -> Path:
7         return self._DATA_ROOT / self.SPORT_FOLDER
8
9     def _load_csv(self, filename, dtype=None, date_cols=None):
10        df = pd.read_csv(self.root / filename, dtype=dtype or {})
11        for col in (date_cols or []):
12            if col in df.columns:
13                df[col] = pd.to_datetime(df[col], errors="coerce")
14        return df
```

Listing 4 – BaseLoader.py – infrastructure commune à tous les loaders

Les modules d’analyse utilisent également un **pattern singleton de chargement paresseux** : les données ne sont chargées qu’au premier appel et mises en cache pour les appels suivants, garantissant de bonnes performances.

7 Tests et validation

7.1 Stratégie de test

Le projet adopte une stratégie de tests unitaires rigoureuse avec pytest. Conformément aux recommandations du README, les tests n’utilisent jamais les données réelles (fichiers CSV) mais des **DataFrames synthétiques** créés dans des fixtures pytest. Cela garantit des tests rapides, reproductibles et indépendants de l’environnement de données.

Les modules de test sont organisés en miroir du code source. Chaque module d’analyse sport dispose d’un `monkeypatch` des variables globales (`_players`, `_teams`, `_matches`) avec des données synthétiques via les fixtures. Chaque test vérifie non seulement le résultat attendu, mais aussi les cas d’erreur (`ValueError` pour les entrées invalides, gestion des données manquantes, etc.).

7.2 Couverture

Package	Fichiers de test	Lignes de test
Model	5	984
Analysis	10	2 188
Parsers	1	531
Common	1	93
Total	17	3 796

Les tests se lancent via :

```
python -m pytest --cov=src --cov-report=term-missing
```

7.3 Formatage et linting

L'ensemble du code est formaté avec **black** (version 25.1.0) et vérifié avec **Flake8**. Le respect de ces outils garantit une cohérence stylistique sur l'ensemble du projet et est contrôlé systématiquement avant chaque merge.

8 Résultats

L'application finale permet d'explorer les statistiques de neuf sports à travers une interface CLI complète et fonctionnelle. Nous présentons ci-dessous cinq scénarios d'utilisation représentatifs, illustrés par des captures d'écran.

8.1 Scénario 1 : Navigation et filtrage des sports

Scénario 1 — Navigation et filtrage

L'utilisateur lance l'application et arrive sur le menu principal listant les neuf sports. Il active le filtre *Collectif* + *E-sport* : seuls League of Legends et CS2 apparaissent. Il sélectionne CS2, puis *Équipe*, puis *Classement* pour obtenir le classement de la phase de groupes, affiché sous forme de tableau trié (équipe, victoires, défaites, matchs joués, points).

```
onyxia@vscode-python-740523-0:~/work$ /opt/python/bin/python /home/onyxia/work/Projet-1A-2026/__main__.py
Stats Sports - Menu principal
Filtres actifs : Collectif, E-sport
[0] Filtres
[1] League of Legends
[2] CS2
[+] Suivant | [q] Quitter
```

FIGURE 5 – Menu principal avec filtres actifs (Collectif + E-sport)

8.2 Scénario 2 : Fiche joueur avec sélecteur paginé

Scénario 2 — Fiche joueur avec sélecteur paginé

L'utilisateur choisit Tennis > Hommes > Fiche d'un joueur. Le sélecteur paginé affiche les 443 joueurs ATP par pages de 20. Il utilise le filtre *Pays (IOC)* pour ne voir que les joueurs français, puis sélectionne un joueur. La fiche affiche le nom complet, la nationalité, la date de naissance, la taille, le nombre de victoires et défaites, et les performances par surface.

```

Sélectionnez un joueur (Tennis ATP)

Filtres actifs : Pays (IOC): FRA
32 résultats – Page 1/2

[1] Adrian Mannarino
[2] Alexandre Muller
[3] Antoine Escoffier
[4] Arthur Cazaux
[5] Arthur Fils
[6] Arthur Rinderknech
[7] Benjamin Bonzi
[8] Benoit Paire
[9] Constant Lestienne
[10] Corentin Moutet
[11] Enzo Couacaud
[12] Gael Monfils
[13] Gillian Osmond
[14] Giovanni Mpetshi Perricard
[15] Gregoire Barrere
[16] Harold Mayot
[17] Hugo Gaston
[18] Hugo Grenier
[19] Kyrian Jacquet
[20] Luca Van Assche

[>] Suiv. | [f] Filtrer | [r] Reset | [-] Retour | [q] Quitter

```

FIGURE 6 – Sélecteur paginé avec filtre « Pays : FRA » actif

```

player_id  Prénom   Nom Main dominante Date de naissance Nationalité (IOC) Taille (cm)  Nom complet
144895    Corentin Moutet      L      19/04/1999      FRA      180.0 Corentin Moutet

[e] Exporter CSV | [+] Suivant | [-] Retour | [q] Quitter

```

FIGURE 7 – Fiche individuelle d'un joueur ATP

8.3 Scénario 3 : Bracket de tournoi

Scénario 3 — Bracket de tournoi

L'utilisateur sélectionne Football Champions League > Équipe > Bracket. L'application affiche l'arbre du tournoi avec les résultats de chaque tour éliminatoire, reconstruit à partir des données de matchs. Les gagnants sont mis en évidence en vert. Ce bracket est également disponible pour le tennis (Grand Chelem), le badminton et CS2.



FIGURE 8 – Bracket éliminatoire de la Ligue des Champions UEFA

8.4 Scénario 4 : Statistiques d'équipe e-sport

Scénario 4 — Statistiques d'équipe LoL

L'utilisateur choisit League of Legends > Équipe > Stats d'une équipe. Après sélection de G2 Esports dans le sélecteur paginé, l'application affiche les moyennes par match : kills, assists, morts, or total, tourelles, dragons, barons et durée moyenne des parties en minutes.

Équipe	Kills	Assists	Morts	Or total	Tourelles	Dragons	Barons	Durée moy. (min)
G2	15.89	34.67	10.89	60956.33	7.67	2.44	0.67	32.1

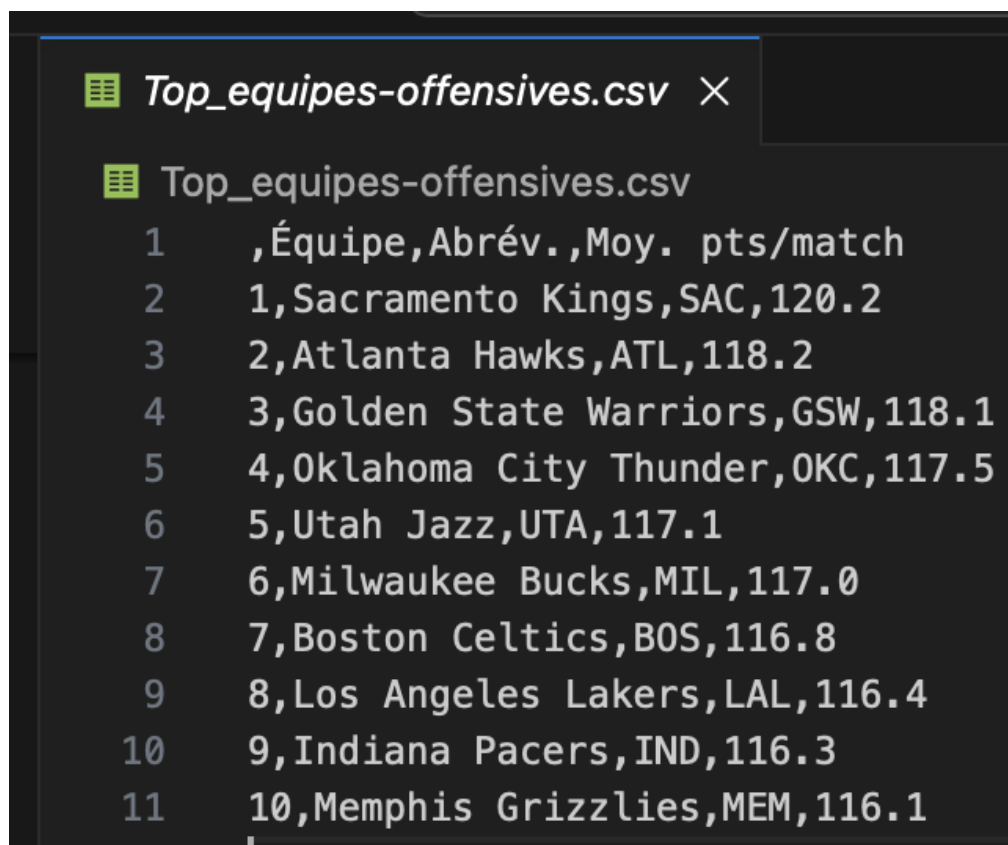
[e] Exporter CSV | [-] Retour | [q] Quitter

FIGURE 9 – Statistiques moyennes par match de G2 Esports (EMEA LEC)

8.5 Scénario 5 : Export CSV

Scénario 5 — Export CSV

Après l’affichage de n’importe quel résultat tabulaire (classement, roster, stats, liste de joueurs), l’utilisateur appuie sur [e] pour exporter le DataFrame en CSV. Un nom de fichier horodaté est proposé par défaut (`{label}_YYYYMMDD_HHMMSS.csv`), encodé en UTF-8-BOM pour compatibilité Excel, et sauvegardé dans le répertoire courant.



```
Top_equipes-offensives.csv X
Top_equipes-offensives.csv
1  ,Équipe,Abrév.,Moy. pts/match
2  1,Sacramento Kings,SAC,120.2
3  2,Atlanta Hawks,ATL,118.2
4  3,Golden State Warriors,GSW,118.1
5  4,Oklahoma City Thunder,OKC,117.5
6  5,Utah Jazz,UTA,117.1
7  6,Milwaukee Bucks,MIL,117.0
8  7,Boston Celtics,BOS,116.8
9  8,Los Angeles Lakers,LAL,116.4
10 9,Indiana Pacers,IND,116.3
11 10,Memphis Grizzlies,MEM,116.1
```

FIGURE 10 – Confirmation d’export CSV après un classement NBA

9 Conclusion

Ce projet a été l’occasion de mettre en pratique les concepts fondamentaux de la programmation Python dans un contexte concret et stimulant. La construction d’une application multi-sports nous a confrontés à des défis techniques variés : concevoir des classes suffisamment génériques pour s’adapter à neuf sports aux structures de données très différentes, normaliser des sources hétérogènes, et maintenir un code modulaire et testable à mesure que le projet grossissait.

Le choix d’une architecture en couches (Parsers → Model → Analysis → CLI) s’est révélé particulièrement pertinent. L’ajout d’un nouveau sport se résume à créer un Loader, un

module d’analyse, et une entrée dans la configuration du `__main__.py` — sans toucher au moteur de navigation ni aux classes génériques. Cette extensibilité nous a permis de passer de cinq à neuf sports sans refactoring majeur.

L’utilisation parcimonieuse des classes, combinée à la puissance de pandas pour le traitement tabulaire, correspond à une approche pragmatique adaptée au contexte du projet. Les classes génériques (**Joueur**, **Equipe**, **Match**) avec leur mapping de colonnes paramétrable offrent un bon compromis entre abstraction et simplicité, comme en témoignent les 3 796 lignes de tests garantissant la fiabilité des composants critiques.

Parmi les améliorations possibles, on pourrait envisager : l’ajout d’une interface graphique (web ou terminal enrichi avec *rich/textual*), l’intégration de visualisations statistiques (matplotlib/seaborn), des fonctionnalités d’analyse comparative entre sports, ou encore la connexion à des API en temps réel. Ce projet nous a en tout cas permis de développer une solide expérience en architecture logicielle, en travail collaboratif via Git, et en exploitation de données structurées avec Python.

9 Références

- [1] McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference.
- [2] van Rossum, G., Warsaw, B., Coghlan, A. (2001). *PEP 8 – Style Guide for Python Code*. Python Software Foundation.
- [3] Krekel, H. et al. (2004–2024). *pytest : helps you write better programs*. <https://docs.pytest.org/>
- [4] Jeu de données NBA 2022–2023. Kaggle (source publique).
- [5] Sackmann, J. *Tennis Abstract — ATP/WTa match results*. <https://www.tennisabstract.com/>
- [6] Liquipedia — base de données e-sport (LoL, CS2, StarCraft II). <https://liquipedia.net/>

Annexes

.0 Annexe A — Bilan personnel de Jade Colin

Bilan personnel

J’ai personnellement trouvé ce projet assez difficile. Au début, j’ai eu besoin de temps pour bien comprendre les attentes ainsi que la manière de démarrer le projet. Étant débutante en programmation, certaines consignes me paraissaient complexes et il n’était pas évident de savoir par où commencer.

De plus, notre groupe était composé de personnes qui ne se connaissaient pas auparavant, ce qui a rendu la cohésion et l’organisation plus difficiles au départ. Malgré cela, nous avons progressivement réussi à communiquer, à échanger nos idées et à trouver une méthode de travail efficace.

Après plusieurs discussions et une meilleure compréhension des objectifs du projet, j’ai commencé à mieux appréhender les différentes étapes de réalisation. Ce projet m’a notamment permis d’approfondir mes connaissances en programmation orientée objet, en apprenant à concevoir des classes et des objets répondant à un besoin concret.

L’une des principales difficultés a été de développer un code générique, capable de fonctionner avec plusieurs bases de données différentes, ce qui demandait une réflexion plus poussée sur l’architecture du programme et sa modularité.

Au final, même si ce projet a été exigeant et parfois difficile, il m’a permis de progresser, de mieux comprendre les principes de la programmation orientée objet et d’améliorer mes compétences techniques ainsi que ma capacité à travailler en équipe.

.0 Annexe B — Bilan personnel de Mathias Douillard

Bilan personnel

Ce projet a été source de difficultés tout au long de sa confection. J’ai trouvé que nous avions très peu d’informations à notre disposition au début du projet, ce qui provient probablement de l’absence de base de données initiale. Il était compliqué de se projeter sur le fonctionnement concret d’une application car on ne savait pas exactement ce qu’elle ferait.

Avec du recul, je comprends que l’objectif derrière cela était que l’on se penche uniquement sur l’architecture du projet et la manière de réfléchir à la création d’une application. Cependant, cela n’était pas assez clair et il était assez compliqué de

bien comprendre les attendus de ce projet. De plus, les informations au sujet des bases de données ainsi que du projet arrivaient au compte-gouttes — que ce soit les attendus ou les barèmes de notation. Cela nous a forcés à modifier des éléments que nous avions déjà conçus.

Enfin, je trouve que les attendus lors des premières séances en autonomie étaient assez élevés. Je sors personnellement de prépa D2 et je découvre la programmation depuis ce début d'année. En dehors de l'UE de remise à niveau et de programmation orientée objet, je n'avais jamais codé de ma vie. J'ai donc eu énormément de mal à comprendre ne serait-ce que les notions évoquées lors de ces séances. Combiné aux éléments indiqués plus haut, j'ai rencontré énormément de difficultés tout au long de ce projet.

Malgré tout, je retire de bonnes choses de ce projet. Je considère avoir pris conscience de l'étendue presque infinie des possibilités liées au code. Je ne m'attendais pas à mettre au point une application au cours de ma première année à l'ENSAI et j'en suis satisfait, même si elle n'est évidemment pas parfaite. Je comprends désormais beaucoup mieux le fonctionnement et la structure du code.

.0 Annexe C — Bilan personnel de Mohammed-Adam Lakhri

Bilan personnel

Ce projet m'a permis de découvrir les réalités du développement logiciel : gestion des données, structuration du code par modules, importance des tests. L'utilisation de datasets réels m'a fait prendre conscience que la qualité des données et la cohérence de leur organisation est souvent le premier obstacle à tout travail d'analyse, opéré avant même d'écrire la première ligne de code.

J'ai pris également conscience que l'on ne peut pas se passer d'une bonne organisation de son code dès le départ : une mauvaise architecture en tout début est souvent un problème plus difficile à corriger par la suite. De même, le développement de tests peut se révéler trop court, ce qui donne l'illusion que tout fonctionne alors que des bugs plus subtils peuvent persister.

Plus personnellement, ce projet m'a appris également à mieux décomposer un problème compliqué en sous-tâches, et à ne pas sous-estimer le temps à allouer à la phase de débogage. C'est souvent là où l'on apprend le plus.

.0 Annexe D — Bilan personnel de Mohamed Fouad Chaabane

Bilan personnel

Ce projet a été pour moi une expérience à la fois stimulante et parfois frustrante, ce qui est sans doute le signe qu'on a vraiment appris quelque chose.

Au départ, j'avoue avoir sous-estimé la difficulté. On s'est dit : « on charge des CSV, on fait quelques stats, c'est bon. » La réalité a été bien différente. Le premier vrai obstacle est arrivé dès la phase de conception : comment faire travailler ensemble quatre personnes sur quatre sous-projets sans que tout se casse à l'intégration ? Ce qui m'a le plus marqué, c'est la valeur concrète de la programmation orientée objet. Au début, ça ressemble à beaucoup de code pour un résultat qui semble identique à ce qu'on aurait fait avec des fonctions simples.

J'ai aussi appris l'importance des tests. Si je devais refaire ce projet, je passerais deux fois plus de temps sur la conception avant d'écrire la première ligne de code. On aurait dû définir ensemble, dès le début, les interfaces entre modules — pas les classes entières, juste les signatures des méthodes importantes et les formats de données échangées. Ça nous aurait évité les heures perdues à déboguer des imports cassés ou des attributs qui n'existaient pas sous le bon nom.

Ce que je retiens surtout, c'est que le travail en équipe sur du code demande presque autant de communication que de programmation. Les outils Git, les classes abstraites et les tests ne font que soutenir cette communication. Ils ne la remplacent pas.